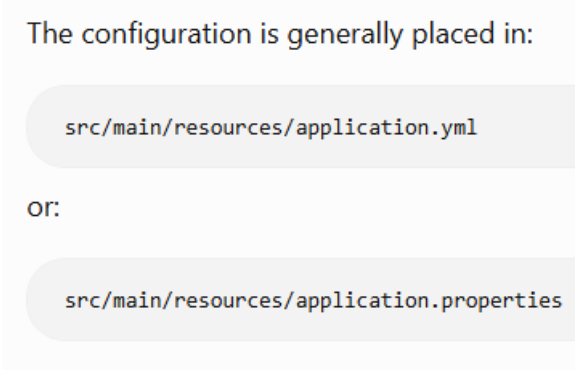
Configure SpringBoot to Work with PostgreSQL

1. Why do we need configuration?

- Adding PostgreSQL is alone not enough
- SpringBoot knows that the application uses PostgreSQL because of the dependency
- But still we need to tell SpringBoot:
  - **Where** is the database?
  - **Which** database should it connect to?
  - **What** username should it use?
  - What password should it use
  - **How** should JPA/Hibernate interact with PostgreSQL



2. application.yml Vs application.properties

YAML

```yaml
spring:
  datasource:
    ...
```

Properties

```properties
spring.datasource...
```

3. Configure the PostgreSQL Data Source

The main configuration is:

YAML

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ecom_db
    username: your_username
    password: your_password
```

4. What is the URL?

```
jdbc:postgresql://localhost:5432/ecom_db
```

Break it down:

```
jdbc
  ↓
Protocol

postgresql
  ↓
Database type

localhost
  ↓
Database host

5432
  ↓
PostgreSQL port

ecom_db
  ↓
Database name
```

5. Understanding JDBC

- jdbc:postgresql://localhost:5432/ecom_db
- **JDBC** – Java Database Connectivity
- It is the **mechanism** used by JAVA applications **to communicate** with **databases**
- PostgreSql tells JDBC which database type/driver is being used
- PostgreSQL's default port is 5432

6. Database Name & Password

You also need to specify the PostgreSQL credentials:

YAML

```yaml
spring:
  datasource:
    username: your_username
    password: your_password
```

- `username` → PostgreSQL user
- `password` → password for that user

The values depend on how PostgreSQL was installed/configured on your machine.

7. JPA Configuration

YAML

```yaml
spring:
  jpa:
    database: postgresql
    show-sql: true
    hibernate:
      ddl-auto: update
```

8. Show-sql

‹/› YAML

```yaml
show-sql: true
```

- Displays the SQL generated by Hibernate/JPA in the console.

For example, when creating a product, you may see something like:

‹/› SQL

```sql
insert into products ...
```

This is particularly useful during **development and debugging**.

9. DDL-auto

YAML

```yaml
hibernate:
  ddl-auto: update
```

- Controls how Hibernate manages the database schema.

`update`

YAML

```yaml
ddl-auto: update
```

- Hibernate attempts to update the database schema based on entity changes.
- Existing data is intended to be preserved.

For example:

```
Java Entity changes
        ↓
Hibernate detects changes
        ↓
Database schema updated
```

10. Create-drop

The lecture also mentions:

```yaml
ddl-auto: create-drop
```

With this setting:

- Hibernate creates the schema when the application starts.
- The schema is dropped when the application stops.
- This can result in **loss of database data**.

Therefore:

```
update
  ↓
Generally preserves existing data

create-drop
  ↓
Schema recreated / data can be lost
```

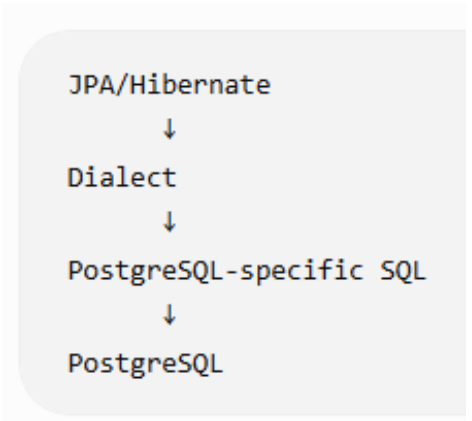
⚠️ **Important:** Be especially careful with destructive DDL settings in environments containing real data.

11. PostgreSQL Dialect

- Different databases can have slightly **different SQL syntax** and capabilities.
- The **dialect** tells **Hibernate** how to **generate SQL appropriate for** the **target database**

YAML

```yaml
spring:
  jpa:
    database-platform: org.hibernate.dialect.PostgreSQLDialect
```



12. Complete configuration

YAML

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ecom_db
    username: your_username
    password: your_password

  jpa:
    database: postgresql
    show-sql: true
    hibernate:
      ddl-auto: update
    database-platform: org.hibernate.dialect.PostgreSQLDialect
```

> Replace `ecom_db`, username, and password with the values for your PostgreSQL setup.

13. Same configuration using application.properties

properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ecom_db
spring.datasource.username=your_username
spring.datasource.password=your_password

spring.jpa.database=postgresql
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

Easy rule

```
YAML:
key:
  child:
    value

Properties:
key.child=value
```

14. What we have configured?

Property	Purpose
`spring.datasource.url`	Where PostgreSQL is located
`spring.datasource.username`	Database username
`spring.datasource.password`	Database password
`spring.jpa.database`	Database type
`spring.jpa.show-sql`	Show generated SQL
`spring.jpa.hibernate.ddl-auto`	Schema management
`spring.jpa.database-platform`	Hibernate database dialect

15. Interview Answer

Question: How do you configure a SpringBoot application to connect to PostgreSQL?

- First, I add the **PostgreSQL JDBC Driver dependency** to the SpringBoot project.
- Then I configure the **datasource URL, username, and password** in application.yml or application.properties
- I also configure **JPA/Hibernate properties** such as **ddl-auto, show-sql, PostgreSQL dialect** so that Hibernate can interact with the PostgreSQL database appropriately

Overall flow

